EXPERIMENT 1

AIM: To set up the deep learning programming environment and implement a simple feedforward neural network from scratch in Python.

CODE:

```
import numpy as np
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[1],[1],[0]], dtype=float)
np.random.seed(42)
W1 = np.random.randn(2, 4)
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1)
b2 = np.zeros((1, 1))
lr = 0.1
epochs = 10000
for epoch in range(epochs):
    Z1 = np.dot(X, W1) + b1
    A1 = sigmoid(Z1)
    Z2 = np.dot(A1, W2) + b2
    A2 = sigmoid(Z2)
    loss = np.mean((y - A2) ** 2)
    dA2 = -(y - A2)
    dZ2 = dA2 * sigmoid_derivative(A2)
    dW2 = np.dot(A1.T, dZ2)
    db2 = np.sum(dZ2, axis=0, keepdims=True)
    dA1 = np.dot(dZ2, W2.T)
    dZ1 = dA1 * sigmoid_derivative(A1)
    dW1 = np.dot(X.T, dZ1)
    db1 = np.sum(dZ1, axis=0, keepdims=True)
    W1 -= lr * dW1
    b1 -= lr * db1
    W2 -= lr * dW2
    b2 -= lr * db2
    if epoch % 2000 == 0:
        print(f'Epoch {epoch}, Loss: {loss:.4f}')
print('\n Final Predictions:')
print(np.round(A2))
```

OUTPUT

```
Epoch 0, Loss: 0.2832
Epoch 2000, Loss: 0.2124
Epoch 4000, Loss: 0.0572
Epoch 6000, Loss: 0.0107
Epoch 8000, Loss: 0.0047

 Final Predictions:
[[0.]
 [1.]
 [1.]
 [0.]]
```

EXPERIMENT 2

AIM: Introduction to deep learning frameworks (TensorFlow, PyTorch) and building a Convolutional Neural Network (CNN) for image classification.

CODE:

```
import tensorflow as tf

from tensorflow.keras import layers, models

import numpy as np

import matplotlib.pyplot as plt

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

X_train = X_train.astype('float32') / 255.0

X_test  = X_test.astype('float32') / 255.0

X_train = X_train.reshape(-1, 28, 28, 1)

X_test  = X_test.reshape(-1, 28, 28, 1)

model = models.Sequential([ layers.Conv2D(32, (3,3),activation='relu',input_shape=(28,28,1)),

layers.MaxPooling2D((2,2)),

layers.Conv2D(64, (3,3), activation='relu'),

layers.MaxPooling2D((2,2)),

layers.Flatten(),

layers.Dense(64, activation='relu'),

layers.Dense(10,activation='softmax')])

model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=5, validation_split=0.1, verbose=1)

test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)

print(f'Test Accuracy: {test_acc:.4f}')
```

OUTPUT

```
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 29s 15ms/step - accuracy: 0.8931 - loss: 0.3585 - val_accuracy: 0.9858 - val_loss: 0.0487
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 25s 14ms/step - accuracy: 0.9848 - loss: 0.0486 - val_accuracy: 0.9878 - val_loss: 0.0429
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 25s 15ms/step - accuracy: 0.9911 - loss: 0.0292 - val_accuracy: 0.9893 - val_loss: 0.0417
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 25s 15ms/step - accuracy: 0.9929 - loss: 0.0210 - val_accuracy: 0.9887 - val_loss: 0.0478
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 26s 15ms/step - accuracy: 0.9951 - loss: 0.0154 - val_accuracy: 0.9910 - val_loss: 0.0387
Test Accuracy: 0.9890
```

EXPERIMENT 3

AIM: Implementing a Recurrent Neural Network (RNN) for sequence modeling — predicting the next value in a sine wave time series.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
t = np.linspace(0, 100, 1000)
data = np.sin(t)
window_size = 50

X, y = [], []

for i in range(len(data) – window_size):

X.append(data[i:i+window_size])

y.append(data[i+window_size])

X = np.array(X).reshape(-1, window_size, 1)

y = np.array(y)

split = int(len(X) * 0.8)

X_train, X_test = X[:split], X[split:]

y_train, y_test = y[:split], y[split:]

model = models.Sequential([

layers.LSTM(64, activation='tanh', input_shape=(window_size, 1), return_sequences=True),

layers.LSTM(32, activation='tanh'),

layers.Dense(1)])

model.compile(optimizer='adam', loss='mse')

history = model.fit(X_train, y_train, epochs=10,batch_size=32, validation_split=0.1, verbose=1)

predictions = model.predict(X_test)
```

OUTPUT

```
Epoch 1/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 12s 138ms/step - loss: 0.3101 - val_loss: 0.0090
Epoch 2/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 84ms/step - loss: 0.0138 - val_loss: 0.0039
Epoch 3/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 84ms/step - loss: 0.0026 - val_loss: 8.9491e-04
Epoch 4/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 7.1616e-04 - val_loss: 3.3982e-04
Epoch 5/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 84ms/step - loss: 3.6394e-04 - val_loss: 2.5293e-04
Epoch 6/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 1.9803e-04 - val_loss: 1.2013e-04
Epoch 7/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 1.1031e-04 - val_loss: 7.2702e-05
Epoch 8/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 5.7917e-05 - val_loss: 5.0014e-05
Epoch 9/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 83ms/step - loss: 3.8631e-05 - val_loss: 2.2166e-05
Epoch 10/10
22/22 ━━━━━━━━━━━━━━━━━━━━ 2s 82ms/step - loss: 1.9065e-05 - val_loss: 1.2213e-05
6/6 ━━━━━━━━━━━━━━━━━━━━ 2s 205ms/step
```

EXPERIMENT 4

AIM: To train neural networks using Stochastic Gradient Descent (SGD) and implement advanced optimization techniques (Adam optimizer) along with regularization methods: Dropout and Weight Decay (L2).

CODE:

```
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
import matplotlib.pyplot as plt
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
X_train = X_train.reshape(-1, 784).astype("float32") / 255.0
X_test = X_test.reshape(-1, 784).astype("float32") / 255.0
model_sgd = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='soft     max')])
model_sgd.compile( optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model_adam = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')])
model_adam.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
model_reg = models.Sequential([ layers.Input(shape=(784,)),
    layers.Dense(128,activation='relu',kernel_regularizer=regularizers.l2(0.001)),
    layers.Dropout(0.3),
    layers.Dense(64,activation='relu',kernel_regularizer=regularizers.l2(0.001) ),
    layers.Dropout(0.3),
    layers.Dense(10, activation='softmax')])
model_reg.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
h_sgd = model_sgd.fit(X_train, y_train,epochs=5,validation_split=0.1,verbose=1)
h_adam = model_adam.fit(X_train, y_train,epochs=5,validation_split=0.1,verbose=1)
h_reg = model_reg.fit(X_train, y_train,epochs=5,validation_split=0.1,verbose=1)
print("\nSGD Test Accuracy:")
print(model_sgd.evaluate(X_test, y_test, verbose=0)[1])
print("\nAdam Test Accuracy:")
```

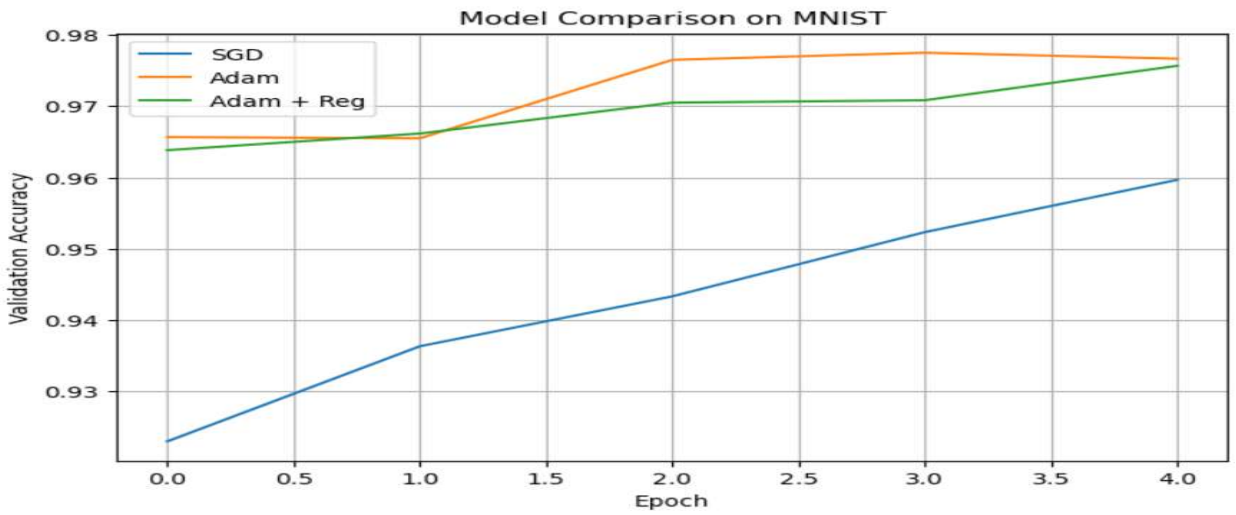
```
print(model_adam.evaluate(X_test, y_test, verbose=0)[1])
print("\nAdam + Regularization Test Accuracy:")
print(model_reg.evaluate(X_test, y_test, verbose=0)[1])
plt.figure(figsize=(8, 5))
plt.plot(h_sgd.history['val_accuracy'], label='SGD')
plt.plot(h_adam.history['val_accuracy'], label='Adam')
plt.plot(h_reg.history['val_accuracy'], label='Adam + Reg')
plt.xlabel('Epoch')
plt.ylabel('Validation Accuracy')
plt.title('Model Comparison on MNIST')
plt.legend()
plt.grid(True)
plt.show()
```

OUTPUT

```
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.7121 - loss: 1.0640 - val_accuracy: 0.9230 - val_loss: 0.2778
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9099 - loss: 0.3141 - val_accuracy: 0.9363 - val_loss: 0.2277
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9204 - loss: 0.2707 - val_accuracy: 0.9433 - val_loss: 0.1974
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9349 - loss: 0.2239 - val_accuracy: 0.9523 - val_loss: 0.1725
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9426 - loss: 0.1946 - val_accuracy: 0.9597 - val_loss: 0.1532
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8753 - loss: 0.4300 - val_accuracy: 0.9657 - val_loss: 0.1190
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9668 - loss: 0.1098 - val_accuracy: 0.9655 - val_loss: 0.1177
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9783 - loss: 0.0691 - val_accuracy: 0.9765 - val_loss: 0.0795
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9830 - loss: 0.0532 - val_accuracy: 0.9775 - val_loss: 0.0792
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9867 - loss: 0.0394 - val_accuracy: 0.9767 - val_loss: 0.0880
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.7821 - loss: 0.8921 - val_accuracy: 0.9638 - val_loss: 0.2980
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9278 - loss: 0.4103 - val_accuracy: 0.9662 - val_loss: 0.2549
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9386 - loss: 0.3477 - val_accuracy: 0.9705 - val_loss: 0.2328
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9457 - loss: 0.3236 - val_accuracy: 0.9708 - val_loss: 0.2264
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9473 - loss: 0.3108 - val_accuracy: 0.9757 - val_loss: 0.2147
SGD Test Accuracy:
0.9480000138282776

Adam Test Accuracy:
0.9754999876022339

Adam + Regularization Test Accuracy:
0.965499997138977
```



EXPERIMENT 5

AIM: To implement Transfer Learning with a pre-trained model (VGG16 trained on ImageNet) and fine-tune it for a domain-specific classification task.

CODE:

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras import layers, models
import numpy as np
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.cifar10.load_data()
X_train = X_train.astype('float32') / 255.0
X_test  = X_test.astype('float32') / 255.0
y_train_oh = tf.keras.utils.to_categorical(y_train, 10)
y_test_oh  = tf.keras.utils.to_categorical(y_test,  10)
def resize_images(images):
return tf.image.resize(images, [48, 48])
X_train_resized = resize_images(X_train)
X_test_resized  = resize_images(X_test)
base_model = VGG16(weights='imagenet',include_top=False,input_shape=(48, 48, 3))
base_model.trainable = False
model = models.Sequential([base_model,
        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.4),
        layers.Dense(10,  activation='softmax')])
model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])
print('--- Phase 1: Feature Extraction ---')
model.fit(X_train_resized, y_train_oh, epochs=5,batch_size=64, validation_split=0.1, verbose=1)
base_model.trainable = True
for layer in base_model.layers[:-4]:
     layer.trainable = False
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),loss='categorical_crossentropy',metrics=['accu
racy'])
print('--- Phase 2: Fine-Tuning ---')
model.fit(X_train_resized, y_train_oh, epochs=5,batch_size=64, validation_split=0.1, verbose=1)
loss, acc = model.evaluate(X_test_resized, y_test_oh, verbose=0)
print(f'Final Test Accuracy: {acc:.4f}')
```

OUTPUT

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 4263s 25us/step
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 468s 8us/step
--- Phase 1: Feature Extraction ---
Epoch 1/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 395s 554ms/step - accuracy: 0.4179 - loss: 1.6524 - val_accuracy: 0.5708 - val_loss: 1.2107
Epoch 2/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 271s 385ms/step - accuracy: 0.5680 - loss: 1.2442 - val_accuracy: 0.5976 - val_loss: 1.1437
Epoch 3/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 166s 235ms/step - accuracy: 0.5900 - loss: 1.1726 - val_accuracy: 0.6106 - val_loss: 1.1047
Epoch 4/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 156s 221ms/step - accuracy: 0.6076 - loss: 1.1281 - val_accuracy: 0.6136 - val_loss: 1.1078
Epoch 5/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 152s 216ms/step - accuracy: 0.6155 - loss: 1.1012 - val_accuracy: 0.6254 - val_loss: 1.0750
--- Phase 2: Fine-Tuning ---
Epoch 1/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 207s 291ms/step - accuracy: 0.6619 - loss: 0.9679 - val_accuracy: 0.7074 - val_loss: 0.8447
Epoch 2/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 204s 290ms/step - accuracy: 0.7351 - loss: 0.7543 - val_accuracy: 0.7264 - val_loss: 0.7789
Epoch 3/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 5558s 8s/step - accuracy: 0.7739 - loss: 0.6410 - val_accuracy: 0.7472 - val_loss: 0.7399
Epoch 4/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 191s 272ms/step - accuracy: 0.8031 - loss: 0.5572 - val_accuracy: 0.7506 - val_loss: 0.7360
Epoch 5/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 205s 292ms/step - accuracy: 0.8281 - loss: 0.4844 - val_accuracy: 0.7606 - val_loss: 0.7101
Final Test Accuracy: 0.7553
```

EXPERIMENT 6

AIM: To implement a Generative Adversarial Network (GAN) for generating handwritten digit images similar to the MNIST dataset.

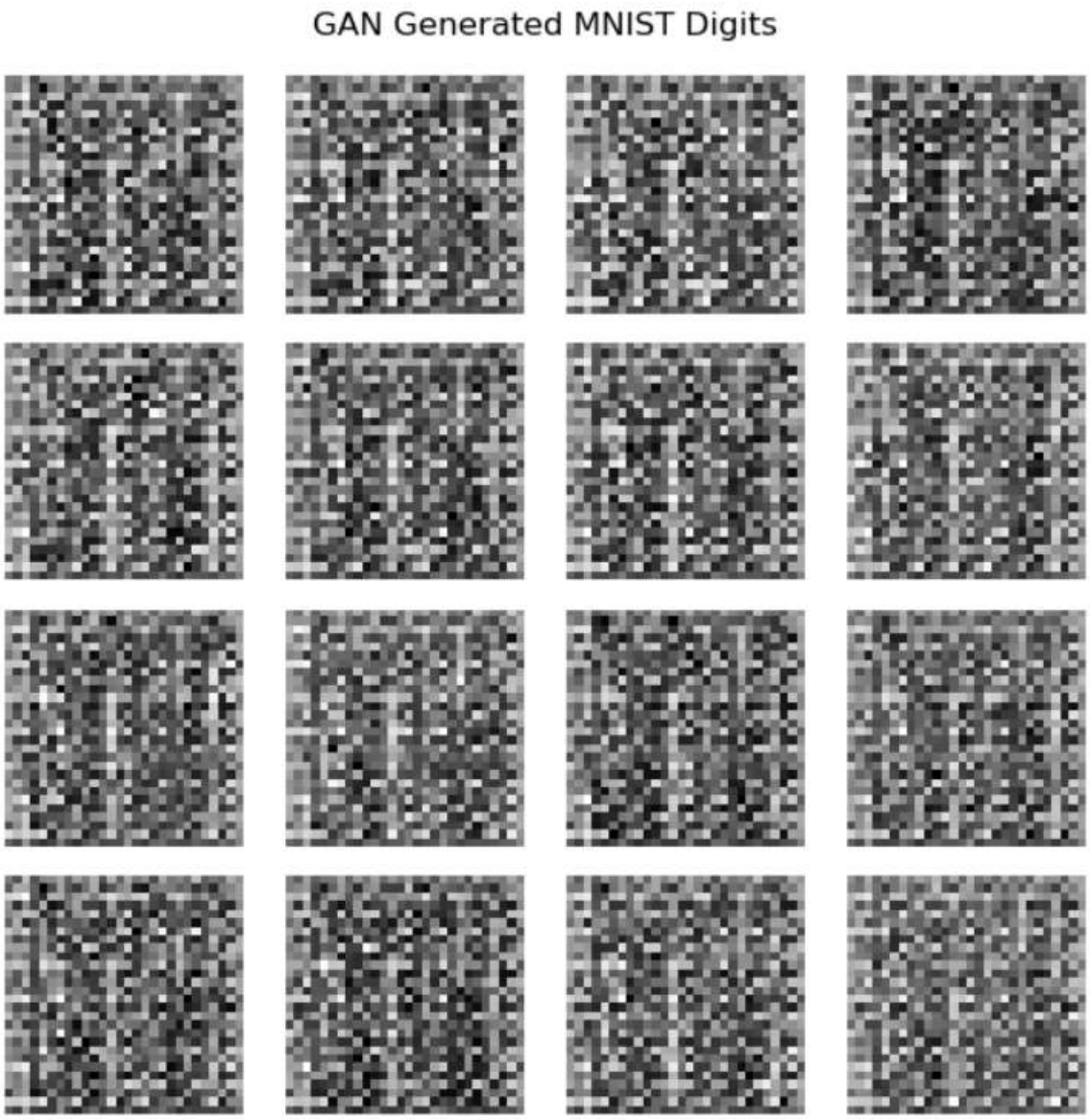
CODE:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
(X_train, _), _ = tf.keras.datasets.mnist.load_data()
X_train = (X_train.astype('float32') - 127.5) / 127.5
X_train = X_train.reshape(-1, 28, 28, 1)
LATENT_DIM = 100 def build_generator():
def build_generator():
    model = models.Sequential([
        layers.Dense(7*7*128, input_dim=LATENT_DIM),
        layers.Reshape((7, 7, 128)),
        layers.BatchNormalization(),
        layers.LeakyReLU(0.2),
        layers.Conv2DTranspose(64, (4,4), strides=(2,2), padding='same'),
        layers.BatchNormalization(),
        layers.LeakyReLU(0.2),
        layers.Conv2DTranspose(1, (4,4), strides=(2,2), padding='same',activation='tanh')])
    return model
def build_discriminator():
  model = models.Sequential([
    layers.Conv2D(64, (3,3), strides=(2,2), padding='same',input_shape=(28,28,1)),
    layers.LeakyReLU(0.2),
    layers.Dropout(0.3),
    layers.Conv2D(128, (3,3), strides=(2,2), padding='same'),
    layers.LeakyReLU(0.2),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(1, activation='sigmoid')])
  return model
generator = build_generator()
discriminator = build_discriminator()
```

```
discriminator.compile(optimizer='adam', loss='binary_crossentropy',metrics=['accuracy'])
discriminator.trainable = False
gan_input  = tf.keras.Input(shape=(LATENT_DIM,))
gan_output = discriminator(generator(gan_input))
gan = tf.keras.Model(gan_input, gan_output)
gan.compile(optimizer='adam', loss='binary_crossentropy')
EPOCHS = 50
BATCH  = 64
for epoch in range(EPOCHS):
    idx = np.random.randint(0, X_train.shape[0], BATCH)
    real_imgs = X_train[idx]
    noise = np.random.randn(BATCH, LATENT_DIM)
    fake_imgs = generator.predict(noise, verbose=0)
    real_labels = np.ones((BATCH, 1))
    fake_labels = np.zeros((BATCH, 1))
    d_loss_real = discriminator.train_on_batch(real_imgs, real_labels)
    d_loss_fake = discriminator.train_on_batch(fake_imgs, fake_labels)
    noise   = np.random.randn(BATCH, LATENT_DIM)
    g_loss  = gan.train_on_batch(noise, np.ones((BATCH, 1)))
    if (epoch+1) % 10 == 0:
        print(f'Epoch {epoch+1}/{EPOCHS} | G Loss: {g_loss:.4f} | D Loss: {d_loss_real[0]:.4f}')
noise = np.random.randn(16, LATENT_DIM)
generated = generator.predict(noise)
generated = (generated * 127.5 + 127.5).astype('uint8')
fig, axes = plt.subplots(4, 4, figsize=(6,6))
for i, ax in enumerate(axes.flat):
   ax.imshow(generated[i].reshape(28,28), cmap='gray')
   ax.axis('off')
plt.suptitle('GAN Generated MNIST Digits')
plt.tight_layout()
plt.show()
```

OUTPUT

```
Epoch 10/50 | G Loss: 0.7138 | D Loss: 0.7130
Epoch 20/50 | G Loss: 0.7217 | D Loss: 0.7207
Epoch 30/50 | G Loss: 0.7301 | D Loss: 0.7290
Epoch 40/50 | G Loss: 0.7398 | D Loss: 0.7385
Epoch 50/50 | G Loss: 0.7503 | D Loss: 0.7488
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 112ms/step
```



EXPERIMENT 7

AIM: To evaluate neural network models using performance metrics, visualize activations and gradients for interpretability, and apply a neural network to a real-world classification problem (heart disease prediction).

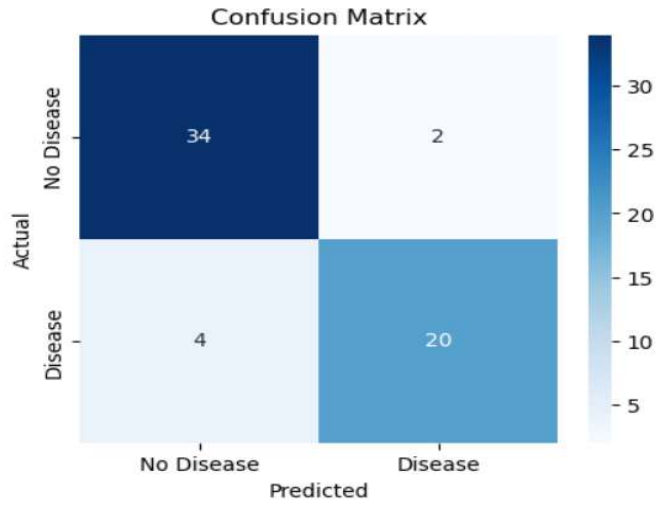
CODE:

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
url = ('https://archive.ics.uci.edu/ml/machine-learning-databases"/heart-disease/processed.cleveland.data')
cols = ['age','sex','cp','trestbps','chol','fbs','restecg','thalach','exang','oldpeak','slope','ca','thal','target']
df = pd.read_csv(url, names=cols, na_values='?').dropna()
df['target'] = (df['target'] > 0).astype(int)
X = df.drop('target', axis=1).values
y = df['target'].values
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)
model = models.Sequential([
   layers.Dense(64, activation='relu', input_shape=(13,)),
   layers.Dropout(0.3),
   layers.Dense(32, activation='relu'),
   layers.Dropout(0.2),
   layers.Dense(16, activation='relu'),
   layers.Dense(1,  activation='sigmoid')])
model.compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy'])
model.summary()
history = model.fit(X_train, y_train, epochs=100,batch_size=16, validation_split=0.15, verbose=0)
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Loss: {loss:.4f}  |  Test Accuracy: {acc:.4f}')
y_pred = (model.predict(X_test) > 0.5).astype(int).flatten()
```

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',xticklabels=['No Disease','Disease'],yticklabels=['No Disease','Disease'])
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
print(classification_report(y_test, y_pred, target_names=['No Disease','Disease']))
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,4))
ax1.plot(history.history['accuracy'],     label='Train')
ax1.plot(history.history['val_accuracy'], label='Validation')
ax1.set_title('Model Accuracy')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()
ax2.plot(history.history['loss'],     label='Train')
ax2.plot(history.history['val_loss'], label='Validation')
ax2.set_title('Model Loss')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
ax2.legend()
plt.tight_layout()
plt.show()
sample = tf.constant(X_test[:1], dtype=tf.float32)
with tf.GradientTape() as tape:
      tape.watch(sample)
       prediction = model(sample)
grads = tape.gradient(prediction, sample).numpy()[0]
feature_names = ['age','sex','cp','trestbps','chol','fbs','restecg','thalach','exang','oldpeak','slope','ca','thal']
plt.figure(figsize=(10,4))
plt.barh(feature_names, np.abs(grads), color='steelblue')
plt.title('Feature Importance via Gradient Saliency')
plt.xlabel('|Gradient|')
plt.tight_layout()
plt.show()
```

OUTPUT

```
Test Loss: 0.4458  |  Test Accuracy: 0.9000
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
```



	precision	recall	f1-score	support
No Disease	0.89	0.94	0.92	36
Disease	0.91	0.83	0.87	24
accuracy			0.90	60
macro avg	0.90	0.89	0.89	60
weighted avg	0.90	0.90	0.90	60